Lesson plan · for the teacher

Memory: the context window

AI agents · Level 3 · Lesson 3

Time
40 min

Materials
Board and chalk, and a duster; A story starter, such as “On the way to the market, Amina saw…”; A corner of the board for short notes

Learning goals

- Explain the context window: a model’s only memory.
- Predict what an agent forgets when the window fills.
- Choose what to keep: short notes, fewer turns, or a new conversation.

Activity: The ten-word window

1. Warm-up (5 min): Ask pupils to repeat, word for word, the first thing you said in the lesson. Most can give the gist but not the words: memory has limits.
2. Teach (8 min): Explain that a model reads the whole text again every time, but only as much as fits its window (Darsi Tiny: 384 tokens; big cloud models: about 128,000). What falls out is gone.
3. Ten-word window (15 min): Write a story starter. Pupils add one word each; after each word, rub out the oldest so only ten stay. After twenty words, ask about the beginning.
4. Notes (7 min): Play again, with one pupil keeping a short line of key facts in a corner of the board. Ask the same questions and compare.
5. Wrap-up (5 min): Ask what an app should keep for an agent, and what it should never keep (pupils’ private details). Then give the exit ticket.

You only need the board, chalk and a duster. Choose a story starter about a place your pupils know.

Discussion

- When has a long chat with an agent gone wrong for you?
- What should never go into a conversation with an agent?
- Why might a new conversation be the best fix?

Exit ticket

- In one sentence, explain why an agent forgot something, and one way to stop it.

Try it at school tomorrow
Play “The ten-word window”: build a story on the board, one word per pupil, but keep only the last ten words and rub out the rest. Then ask about the beginning: what does the class still know, and what has fallen out?

Practice questions · for you

Memory: the context window

AI agents · Level 3 · Lesson 3

1 **A colleague pastes thirty pupils' reports into one chat with a small agent, then asks about the first pupil. What might go wrong?**

- ◯ The first reports may have fallen out, so the answer could be made up
- ◯ The agent will refuse to read reports
- ◯ Nothing: agents remember everything

2 **Put in order what happened in the memory lab with "Drop the oldest" and a small window.**

- ☐ The app drops the oldest turns to make room
- ☐ You reply "Dala", and it gives today's weather
- ☐ Your question, with "tomorrow" in it, falls out
- ☐ You ask: "What's the weather tomorrow?"
- ☐ Darsi Tiny asks back: "Which village?"

3 **Match each way of handling a full window to what happens.**

Keep everything	• • • •	The start of a task can fall out first
Drop the oldest	• • • •	The window fills, and the fourth answer is cut off
Keep notes	• • • •	The window is empty again, and nothing carries over
Start a new conversation	• • • •	Finished requests keep only the question and the answer

4 **This app keeps notes. What does it keep from a finished request?**

```
def note(turn):
    return {"asked": turn["request"],
            "answered": turn["answer"]}
```

- ◯ Only the tool's result
- ◯ Only the question and the answer
- ◯ Everything, including the tool calls and results

5 **When part of a conversation falls out of the context window, the model can still look it up later if it needs it.**

- ◯ True
- ◯ False

✎ **Think of a long task you might do with an agent, such as planning a term. What would you keep in short notes, and what would you never put in the conversation?**

Check your answers

Memory: the context window

AI agents · Level 3 · Lesson 3

1 **The first reports may have fallen out, so the answer could be made up**

The first reports may no longer be in the window, and the model can't know what it never saw.

2 **Top to bottom: 3, 5, 4, 1, 2**

The question came first, so it was the first to fall out when the app made room.

3 **Keep everything — The window fills, and the fourth answer is cut off**
Drop the oldest — The start of a task can fall out first
Keep notes — Finished requests keep only the question and the answer
Start a new conversation — The window is empty again, and nothing carries over

Each choice trades memory for room: everything fills up, dropping loses the start, notes keep what matters in fewer tokens, and a new conversation starts clean.

4 **Only the question and the answer**

The note keeps two things: what was asked (the request) and what was answered.

5 **False**

The window is the model's only memory: what falls out is gone for it.